

Modelowanie struktury: diagramy klas

dr hab. Michał Śmiałek, prof. uczelni
dr Kamil Rybiński



**Wydział
Elektryczny**
POLITECHNIKA WARSZAWSKA

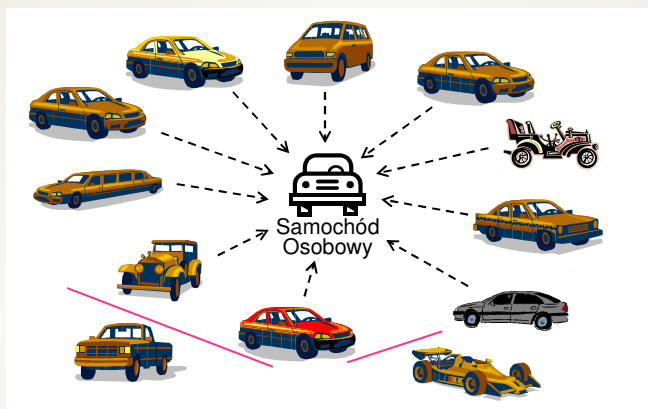
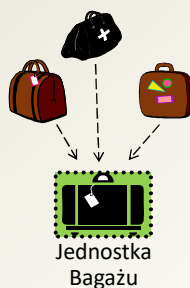
1DI1306: Modelowanie oprogramowania w języku UML



1

Przypomnienie: klasy podobnych obiektów

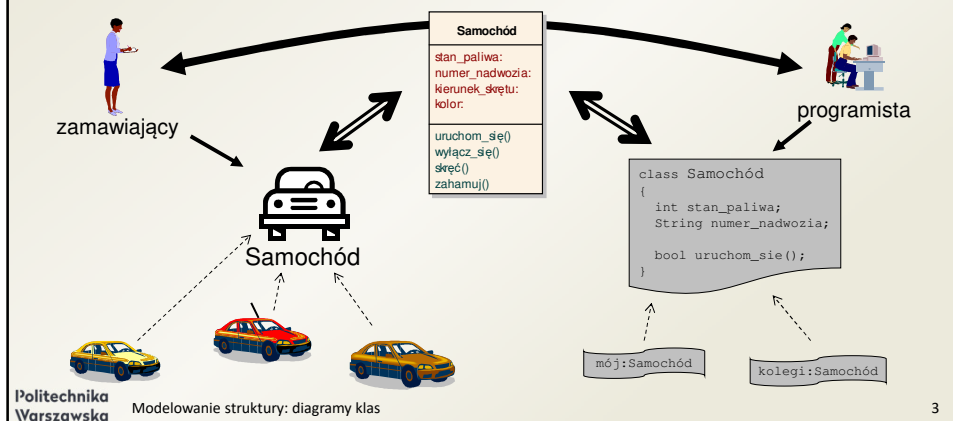
- Klasa: opis grupy obiektów o jednolitym zbiorze atrybutów i sposobie zachowania
 - Jednolity wzorzec dla obiektów (tzw. instancji)



2

Klasa w modelach rzeczywistości lub kodu

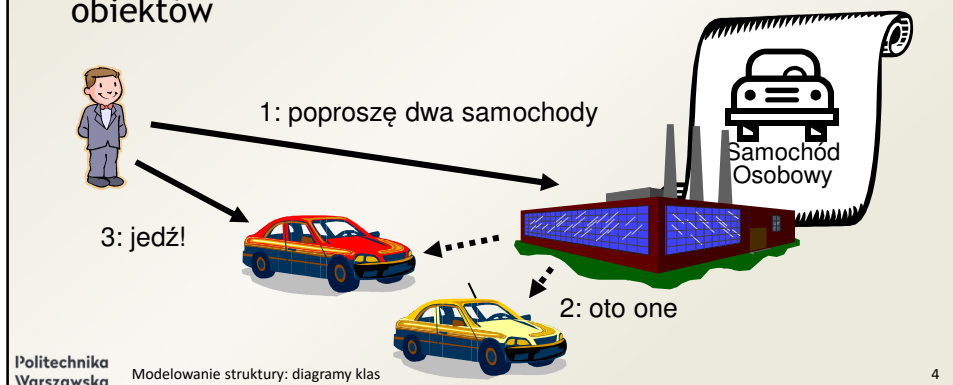
- Klasa jako element opisu rzeczywistości
- Klasa jako element składowy programu
- Ważne: przejście od opisu rzeczywistości do opisu kodu nie jest proste (1 do 1)



3

Klasa jako wzorzec obiektów

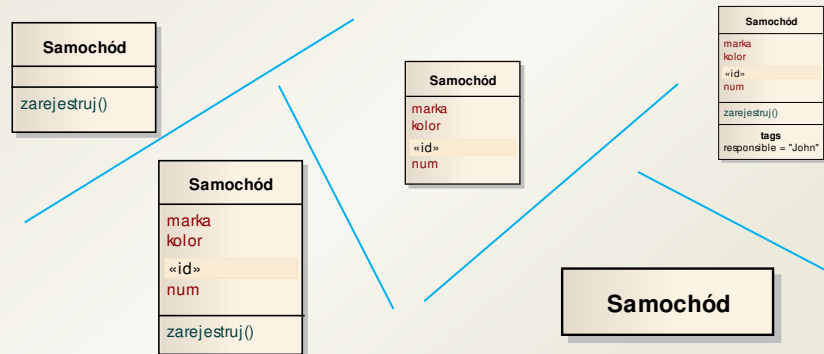
- Klasa jest „planem budowy” obiektów pewnego rodzaju
- Konstrukcja obiektów następuje na prośbę obiektów zewnętrznych
- Po „wyprodukowaniu” można korzystać z usług obiektów



4

UML: reprezentacja klasy

- Klasa jest oznaczana jako prostokąt z nazwą
- Ikona klasy zawiera 1 lub więcej (najczęściej do 3) przegródek: m.in. nazwa i atrybuty
- Na danym diagramie może być ujawnione więcej lub mniej szczegółów (zasada abstrakcji!)



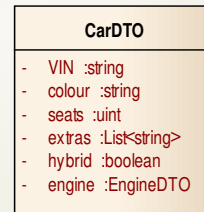
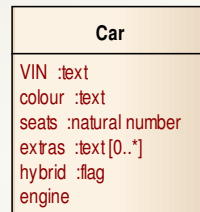
Atrybuty

- Atrybut - element danych zawarty całkowicie i kontrolowany przez odpowiednią klasę
- Składnia atrybutów
 - „nazwa_atrybutu : typ_atrybutu [krotność]”
 - Typ atrybutu jest nieobowiązkowy
 - Nazwy typów nie są w UML ustandaryzowane
- Dobre praktyki modelowania atrybutów
 - Model rzeczywistości - nazwy atrybutów i ich typów zrozumiałe dla klienta (np. w języku polskim)
 - Model kodu - nazwy typów dopasowane do docelowego języka programowania
- Jako typu atrybutu można użyć nazwę klasy
 - Klasy stanowią typy danych i można ich używać tak, jak typów prostych

Atrybuty - przykłady

- Model rzeczywistości

- Zrozumiałe nazwy typów
- Spacje w nazwach
- Możliwy brak typu



```
public class CarDTO {
    private string VIN;
    private string colour;
    private uint seats;
    private List<string> extras;
    private boolean hybrid;
    private EngineDTO engine;
}
```

- Model kodu

- Nazwy typów dostosowane do wybranego języka programowania
- Brak spacji w nazwach
- Atrybuty publiczne i prywatne (+ / -)

Typy wyliczeniowe

- Typ wyliczeniowy - typ danych w postaci ustalonego zbioru wartości (napisów)
- Składnia typów wyliczeniowych
 - Prostokąt z nazwą
 - Nad nazwą oznaczenie «enumeration»
 - Pod nazwą lista literałów (napisów)
- Po zdefiniowaniu, typy wyliczeniowe można używać jak typy proste

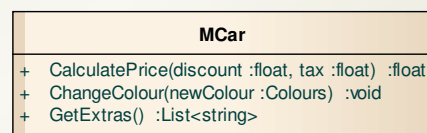
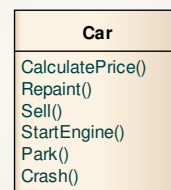


Operacje

- Operacja - elementarna usługa dostarczana przez obiekty danej klasy innym obiektom
 - Rzadko modelowane w ramach opisu rzeczywistości
 - Kluczowe dla modelowania kodu
- Składnia operacji
 - „nazwa_operacji (lista_parametrów) : zwracany_typ”
 - lista_parameterów (sygnatura) zawiera parametry rozdzielone przecinkami
 - Notacja dla parametrów taka jak dla atrybutów
 - zwracany_typ - składnia jak dla typów atrybutów

Operacje - przykłady

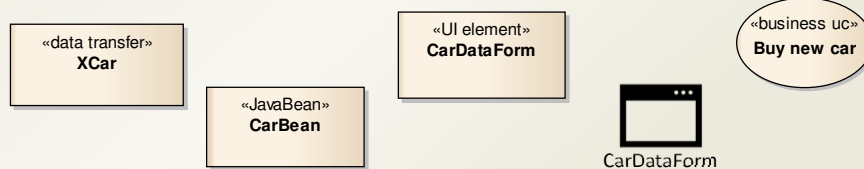
- Model rzeczywistości
 - Zazwyczaj bez parametrów
 - Odpowiadają sposobom wykorzystania obiektów w realnym świecie
- Model kodu
 - Konwencja nazewnicza właściwa dla danego języka programowania
 - Pełne sygnatury operacji
 - Operacje publiczne i prywatne (+ / -)



```
public class MCar {
    public float CalculatePrice(float discount, float tax){
        /* ... */ return price;
    }
    public void ChangeColour(Colours newColour){
        /* ... */
    }
    public List<string> GetExtras(){
        /* ... */ return extras;
    }
}
```

Stereotypy

- Stereotyp umożliwia dostosowanie elementów języka UML do specyfiki danej platformy lub dziedziny
 - Rozszerzenie lub doprecyzowanie znaczenia
 - Dostosowanie do danej technologii
- Notacja stereotypów
 - Nazwa umieszczona w nawiasach kątowych (tzw. guillemet)
 - Stereotyp umieszczamy nad nazwą danego elementu
 - Nadanie stereotypu może powodować zmianę reprezentacji (kształtu, koloru,...) elementu
- Stereotypy można nadawać wszystkim elementom języka UML (nie tylko klasom)



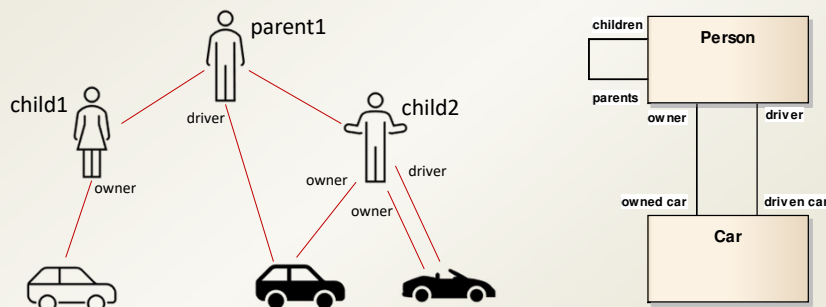
Politechnika
Warszawska Modelowanie struktury: diagramy klas

11

11

Asocjacje między klasami

- Asocjacja definiuje możliwe relacje między obiektami klas (m.in. między obiektami tej samej klasy)
- Klasy mogą pełnić role w asocjacji
 - Role zaznaczamy jako nazwy końców asocjacji
- Może istnieć kilka związków między tymi samymi klasami



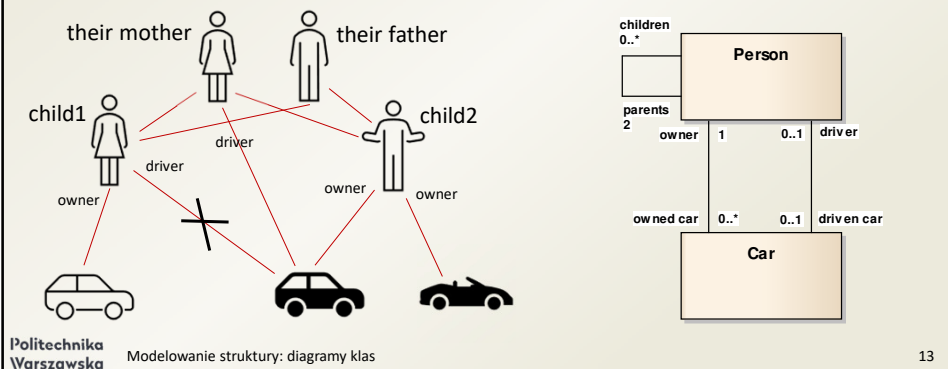
Politechnika
Warszawska Modelowanie struktury: diagramy klas

12

12

Krotności w asocjacjach i dla atrybutów

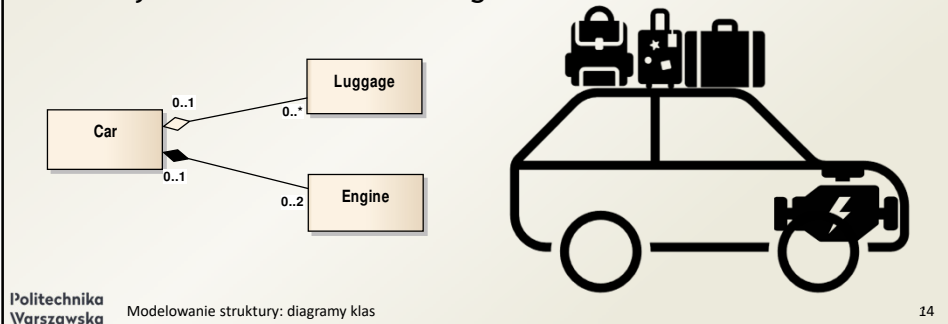
- Krotność - dopuszczalna liczba obiektów klasy w relacji z jednym obiektem drugiej klasy
 - Podajemy dolną i górną granicę zakresu, np. 0..5
 - Górna granica może wynosić nieskończoność (*)
 - Gdy dolna i górna granica są takie same (np. 2..2), wystarczy podać jedną wartość; oznaczenie * jest równoważne 0..*



13

Agregacje i kompozycje

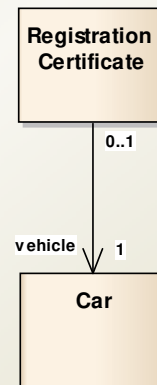
- Agregacja i kompozycja oznaczają asocjacje grupowania, np. grupowania typu całość-części (składowe)
 - WAŻNE, notacja: romb umieszczamy po stronie całości
- W relacji kompozycji składowe nie mogą być dzielone między różne całości
- Kompozycja najczęściej ogranicza czas życia składowych czasem życia całości
- Różnica między agregacją i kompozycją jest czasami płynna i zależy od kontekstu/celu danego modelu



14

Nawigowalność asocjacji

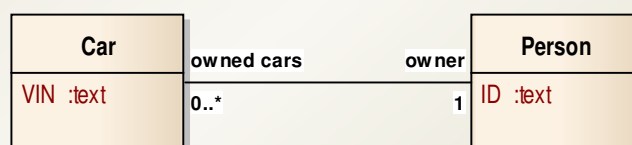
- Możliwość efektywnego osiągnięcia obiektów jednej klasy przez obiekty drugiej klasy
 - Dany obiekt „wie” o innym obiekcie i może np. wykorzystywać jego usługi
- Notacja: asocjacja zakończona strzałką
- Rodzaje nawigowalności
 - Brak nawigowalności
 - Nawigowalność poprzez atrybut
 - Nawigowalność bez atrybutu
- Obecność roli na końcu asocjacji wskazuje na nawigowalność poprzez atrybut



15

Elementarne atrybuty a klasy

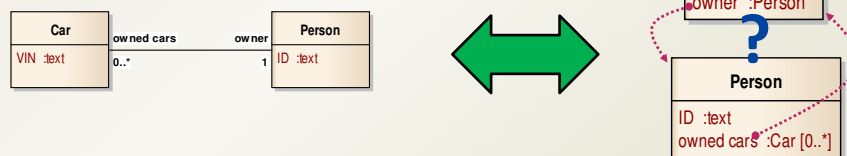
- Przestanki wystarczające do utworzenia klasy
 - Obiekty tego typu posiadają indywidualną tożsamość
 - Obiekty tego typu można zdekomponować na obiekty składowe lub elementarne atrybuty
 - Obiekty tego typu wchodzą w dalsze relacje



16

Asocjacje a atrybuty klas

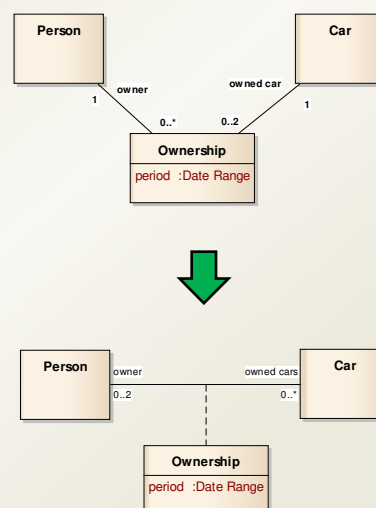
- W języku UML asocjacje mogą łączyć atrybuty klas będących w relacji ze sobą
 - Typ atrybutu odpowiada klasie po drugiej stronie relacji
- Kiedy lepiej stosować asocjacje?
 - Obiekty obydwu klas mają wiedzę nawzajem o sobie
 - Uwypuklenie szczegółów klas będących w relacji
 - Lepsza wizualizacja zależności między obiektami klas
- Kiedy lepiej stosować atrybuty?
 - Nie są istotne szczegóły klas
 - Zawiła sieć połączeń między klasami



17

Klasy asocjacyjne

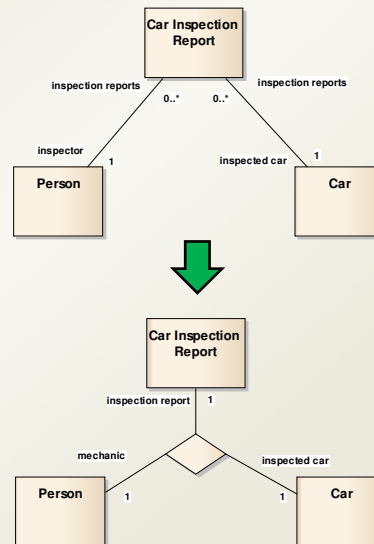
- Asocjacje o cechach klasy
 - Mogą posiadać atrybuty i operacje
- Stosunkowo rzadko stosowane w modelowaniu kodu
 - Mniej oczywiste przełożenie na kod
- Czasami bardziej naturalne w modelu rzeczywistości



18

Asocjacje wielokrotne

- Asocjacja może łączyć wiele klas jednocześnie
- Krotności pisane z punktu widzenia uczestnictwa obiektów w relacji
- Notacja: romb
- Rzadko używane
 - Trudne i niejednoznaczne przełożenie na kod
 - Niezbyt naturalne w typowych zastosowaniach modelowania obiektowego



Politechnika
Warszawska

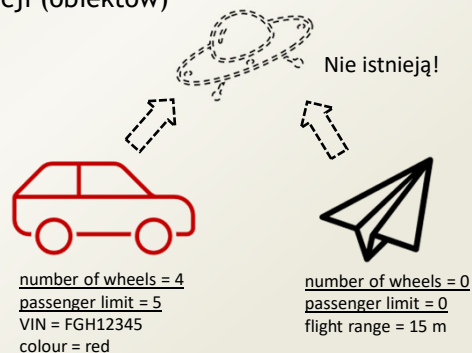
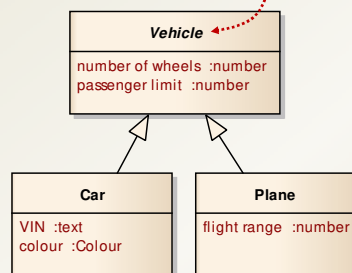
Modelowanie struktury: diagramy klas

19

19

Generalizacje i specjalizacje

- Taksonomia klas obiektów (np. taksonomia świata zwierzęcego): klasy ogólne i klasy specjalizowane
- Klasy „dziedziczą” atrybuty i operacje od klas specjalizowanych; obiekty „sumują” atrybuty i operacje
- Klasy abstrakcyjne
 - Nie posiadają swoich instancji (obiektów)
 - Nazwa pisana kursywą



Politechnika
Warszawska

Modelowanie struktury: diagramy klas

20

20

Ciekawostka: różne notacje klasy

- W początkowym okresie (lata '80 i '90) powstało wiele różnych notacji modelowania
- Notacje zostały ujednolicone w postaci języka UML
- Klasa zapisana w języku programowania też może być traktowana jako model

```
class Konto {
    private:
        int Stan;
    public:
        void Wplata();
}
```



MOSES



Booch



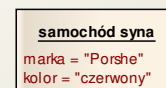
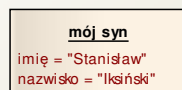
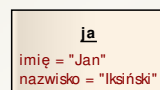
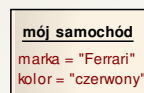
Coad-Yourdon



UML

UML: notacja obiektów

- Obiekt jest reprezentowany jako prostokąt z nazwą
- Nazwa obiektu MUSI być podkreślona
- Nazwa obiektu może mieć spację (jeśli modelujemy rzeczywistość)
- Stan obiektu określany jest poprzez listę wartości atrybutów



Wyróżnianie tożsamości obiektów

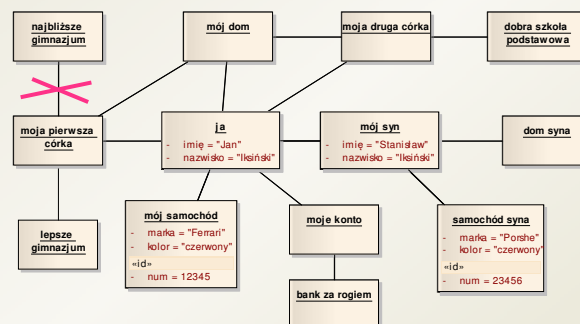
- Wyróżniamy jeden z atrybutów obiektu
- Nadajemy stereotyp, np. «id»
- Wartości tego atrybutu **MUSZĄ** być
 - Unikalne pomiędzy różnymi obiektami
 - Niezmienne od początku istnienia obiektu



23

UML: relacje między obiektami

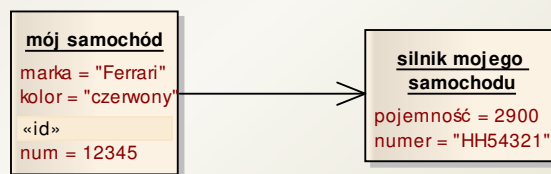
- Relacje zaznaczamy za pomocą łączników
- Obiekty z łącznikami tworzą diagramy obiektów: „fotografia rzeczywistości w danym momencie”



24

UML: relacje zawierania

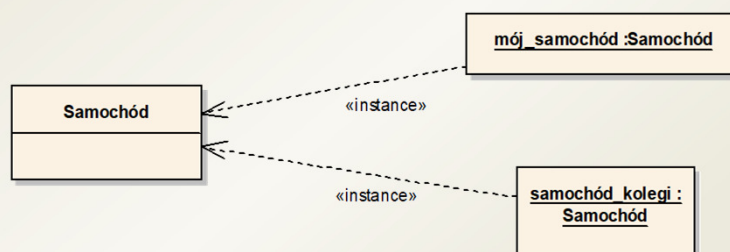
- Obiekty mogą się w sobie zawierać
- Zawieranie obiektów nie jest specjalnie wyróżniane
 - Łącznik dla zawierania może być oznaczony np. dodatkowym grotem strzałki



25

UML: klasa jako typ dla obiektów

- Klasa jest oznaczana jako prostokąt z nazwą (BEZ podkreślenia)
- Klasa obiektu jest oznaczana po dwukropku (jako typ obiektu)



26

Ćwiczenie

- Proszę narysować diagram klas składający się z klasy „Student” i 4 klas w odpowiednich związkach (asocjacje, agregacje, generalizacje) z klasą student i z odpowiednimi krotnościami oraz rolami
- Proszę narysować diagram obiektów zgodny z diagramem klas, składający się z 6 obiektów i odpowiednich łączników